SRP by making you perform initialization code in addition to using the service you created.

One solution to this problem is to make it somebody else's problem. When writing a service, you can declare your dependencies and let another class fill those dependencies for you. Then your service can focus on the functionality for which it was designed instead of trying to work out how to build its dependencies.

This technique is called *dependency injection* or the *Inversion of Control* (IoC) principle, a well-recognized design pattern that is used extensively. Typically, you'll register the dependencies of your application into a *container,* which you can use to create any service. You can use the container to create both your own custom application services and the framework services used by ASP.NET Core. You must register each service with the container before using it in your application.

NOTE I describe the dependency inversion principle and the IoC container used in ASP.NET Core in detail in chapters 8 and 9.

In an ASP.NET Core application, this registration is performed by using the `Services` property of `WebApplicationBuilder`. Whenever you use a new ASP.NET Core feature in your application, you need to come back to Program.cs and add the necessary services. This task isn't always as arduous as it sounds, typically requiring only a line or two of code to configure your applications.

In listing 3.4 we configured an optional service for the HTTP logging middleware by using the line

```
builder.Services.AddHttpLogging(opts =>
    opts.LoggingFields = HttpLoggingFields.RequestProperties);
```